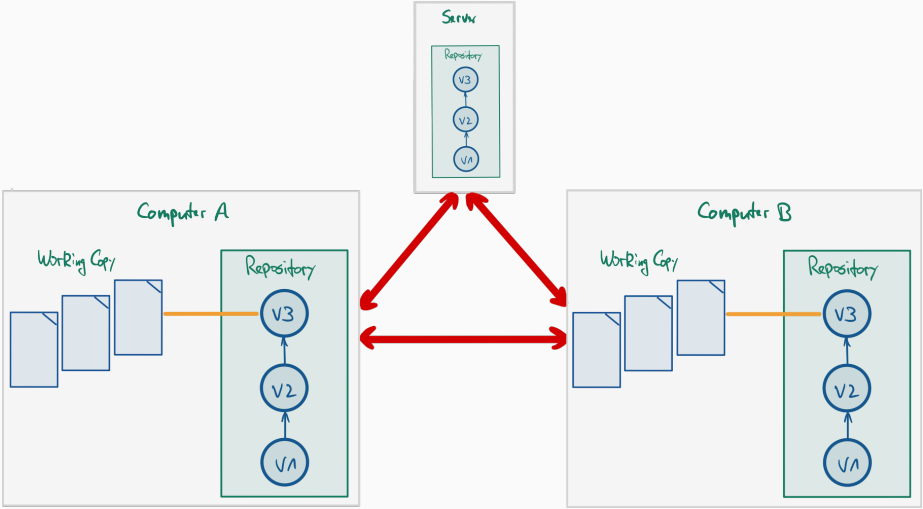
Branching-Strategien & Git-Workflows

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Nutzung von Git in Projekten: Verteiltes Git (und Workflows)



Umgang mit Branches: Themen-Branches

```
                I---J---K  wuppieV1
               /
          D---F  wuppie
         /
A---B---C---E  master
             \
              G---H  test
```

Umgang mit Branches: Langlaufende Branches

```
A---B---D  master
     \
      C---E---I  develop
           \
            F---G---H  topic
```

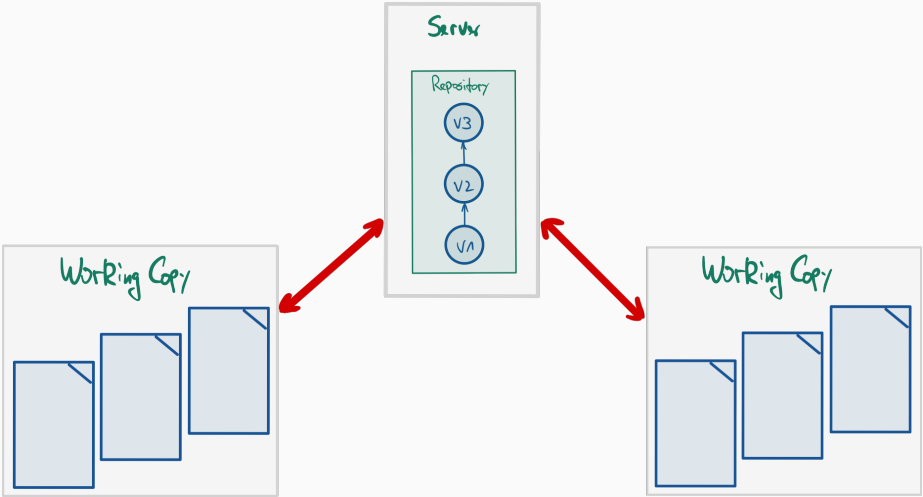
Komplexe Branching-Strategie: Git-Flow

```
A---B--------------------G---J1  master
     \                  / \ /
      \                /   X  fix
       \              /     \
        C-------------F----I--J2  develop
         \           / \  /
          \         /   H1  featureB
           \       /
            D1----D2  featureA
             \
              E1---E2---E3---E4---E5  featureC
```

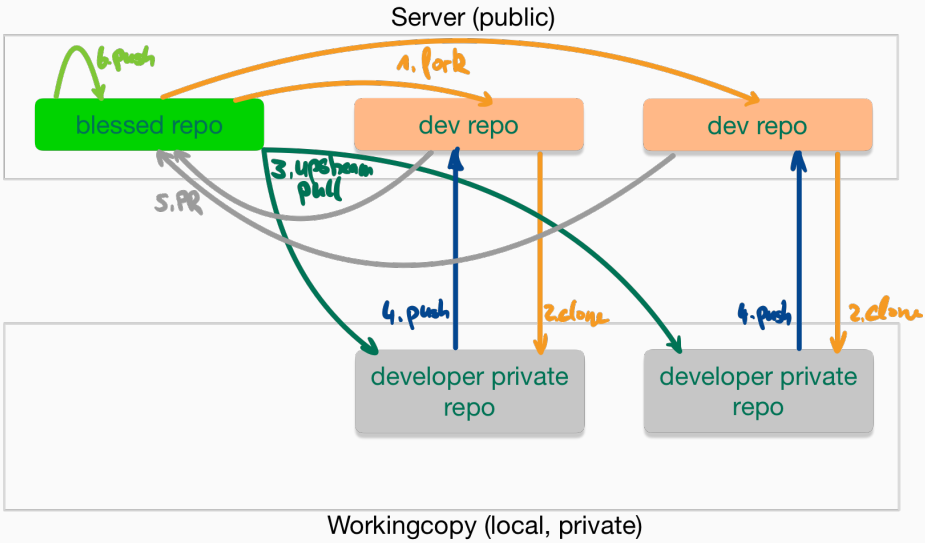
Vereinfachte Braching-Strategie: GitHub Flow

```
A---B---C----D-----------E  master
     \   \  /           /
      \   ta1  topicA  /
       \              /
        tb1---tb2---tb3  topicB
```

Zusammenarbeit: Zentraler Workflow mit Git (analog zu SVN)



Zusammenarbeit: Einfacher verteilter Workflow mit Git



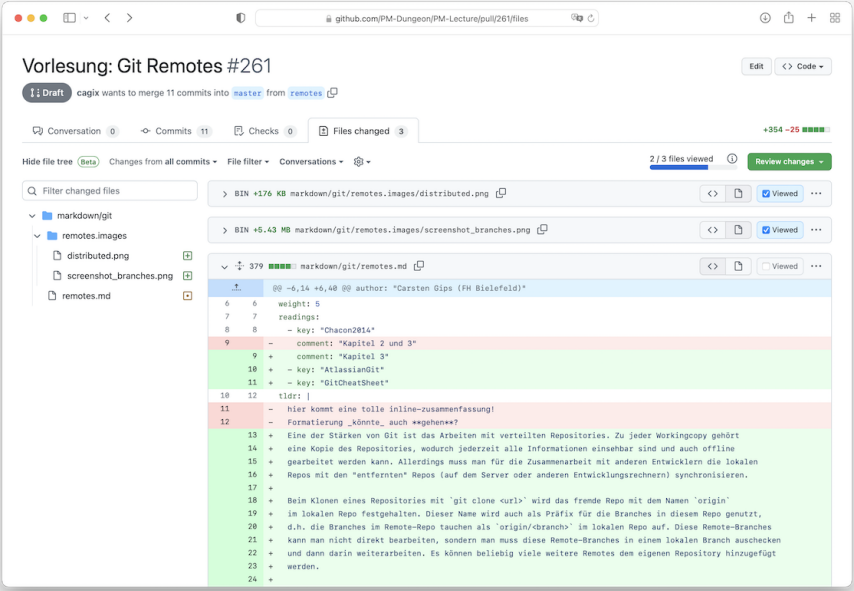
Beispiel: Forken mit Gitlab, Namen für Remotes

Feature-Branches aktualisieren: Mergen mit *master* vs. Rebase auf *master*

```
$ git checkout master          # Workingcopy auf master
$ git pull upstream master     # Aktualisiere lokalen master mit master aus Vorgabe-Repo
$ git checkout feature         # Workingcopy auf feature
$ git merge master             # Aktualisiere feature: Merge master in feature
$ git push origin feature      # Push aktuellen feature ins Team-Repo
```

Besser einen Pull/Rebase für den Feature-Branch machen!

Kommunikation: Merge- bzw. Pull-Requests



Wrap-Up

- Einsatz von Themenbranches für die Entwicklung
- Unterschiedliche Modelle:
  - Git-Flow: umfangreiches Konzept, gut für Entwicklung mit festen Releases
  - GitHub Flow: deutlich schlankeres Konzept, passend für kontinuierliche Entwicklung ohne echte Releases
- Git-Workflows für die Zusammenarbeit:
  - einfacher zentraler Ansatz für kleine Arbeitsgruppen vs.
  - einfacher verteilter Ansatz mit einem “blessed” Repo
- Aktualisieren Ihres Clones und Ihres Forks mit Änderungen aus dem “blessed” Repo
- Erstellen von Beiträgen zu einem Projekt über Pull-/Merge-Requests

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.